

Lab 3 Build Guide

Lab 3 Build Guide: Configuration Generation from Intent

This guide walks through the configuration generation engine that turns the validated data model into per-device FRR configurations. By the end you will have generated complete router configs for all six devices using three different tooling paths and you will understand when each one fits.

What We Built

Labs 1 and 2 established a validated data model. The data exists, it is correct, and it passes 39 checks across four validation layers. But the routers are still unconfigured. Lab 3 closes that gap. It reads the validated intent and produces device-specific FRR configuration files that can be loaded onto each router.

The key concept is intent-to-device translation. The data model says “spine1 is a route reflector.” The config generator turns that into specific BGP neighbor statements, cluster-id configuration, and route-reflector-client designations for every peer. You do not write those neighbor statements by hand. You declare the intent and the automation derives the configuration.

We built three tooling paths to demonstrate that the same intent can be realized with different tools. Path A (Python + Jinja2) is the primary path and the one the pipeline uses. Path B (Ansible) shows how teams already using Ansible would approach the same problem. Path C (Terraform) demonstrates the declarative/stateful approach for comparison.

Prerequisites

You need a working Lab 2 setup with all validation passing. Jinja2 was already added as a dependency by pytest-html, so no new packages are needed.

```
uv run python validate.py
```

All 39 checks should pass.

Part 1: The Render Engine

The render engine lives at `generators/python/render.py`. It does three things: loads the validated data model, builds a per-device template context, and renders Jinja2 templates into FRR configuration files.

```
cat generators/python/render.py
```

Context Building

The most important function in the file is `build_device_context()`. It takes a single device and the complete fabric model, then returns a dict with everything that device's templates need. This is where role-based logic lives.

For BGP neighbors, the logic is straightforward. If the device is a route reflector, it peers with every other device in the fabric and marks non-RR peers as route-reflector-clients. If the device is not a route reflector, it peers only with the route reflectors. This is the auto-peering behavior that LABS.md calls out as a key teaching point. You add a third route reflector to the data model, re-run the generator, and every device's BGP config updates automatically.

For VRFs, spines get nothing (they are fabric-only). Border leafs get all VRFs because they handle external connectivity. Regular leafs get only the VRFs that have network segments matching their assigned VLANs. If leaf1 has ports in VLAN 10 and VLAN 20, it gets the PRODUCTION VRF because those VLANs map to production networks. It does not get the MANAGEMENT VRF because it has no ports in VLAN 900.

For network segments and interfaces, the same filtering applies. Each device only sees the segments and port assignments that are relevant to it.

Running the Generator

```
uv run python -m generators.python.render
```

This produces six configuration files in the `configs/` directory, one per device. The output tells you how many lines each config contains.

Examining the Output

Start with a spine to see the route reflector configuration:

```
cat configs/spine1.conf
```

Spine1's BGP section has `bgp cluster-id 10.0.0.1` and `route-reflector-client` on every non-RR neighbor (leaf1, leaf2, border1, border2). Spine2 gets the same treatment because it is also an RR. The spines peer with each other as regular iBGP peers without the client designation.

Now look at a leaf:

```
cat configs/leaf1.conf
```

Leaf1's BGP section has only two neighbors: spine1 and spine2. No cluster-id, no route-reflector-client. It is a client. Its VRF section contains the BGP EVPN configuration for the PRODUCTION VRF and reference comments documenting the VXLAN segments. The VXLAN dataplane config (vxlan interfaces, bridge mappings, anycast gateways) is platform-specific and would be configured at the OS level on a real switch. In the FRR lab environment we handle the routing plane through vtysh and document the dataplane config as comments.

Now look at a border:

```
cat configs/border1.conf
```

Border1's BGP section is the same as a leaf: two neighbors, both spines. But its VRF section has all three VRFs (PRODUCTION, DEVELOPMENT, MANAGEMENT) and references all five network segments. Borders carry everything because they are the exit point for traffic leaving the fabric.

Compare leaf2 to leaf1:

```
diff configs/leaf1.conf configs/leaf2.conf
```

Different loopbacks, different fabric link IPs, different host-facing interfaces, different VRF assignments. The same intent model, the same templates, completely different device configs.

Part 2: The Jinja2 Templates

The templates live in `generators/python/templates/`. There are five of them, composed through Jinja2's `{% include %}` directive.

frr_base.j2

```
cat generators/python/templates/frr_base.j2
```

This is the entry point. It sets the FRR version, hostname, and logging, then includes the four section templates. The VRF template is conditional: it only renders if the device has VRFs assigned.

frr_interfaces.j2

```
cat generators/python/templates/frr_interfaces.j2
```

Three sections: loopback, fabric links, and host-facing interfaces. The loopback gets its IP address and is placed into OSPF area 0. Fabric links get their IPs and OSPF point-to-point config applied inline. Note that MTU is not set through vtysh because FRR treats that as an OS-level setting (you would use `ip link set` on Linux). Host-facing interfaces get descriptions through vtysh, but switchport and VLAN configuration is platform-specific and handled outside FRR on real switches. The template documents these as comments.

frr_ospf.j2

```
cat generators/python/templates/frr_ospf.j2
```

OSPF configuration with the router-id set to the loopback and reference bandwidth from the data model. The network statements tell OSPF which interfaces to activate. FRR may convert these to interface-level `ip ospf area` commands internally, which is why the loopback also has `ip ospf area` set directly in the interfaces template to ensure it is always advertised.

frr_bgp.j2

```
cat generators/python/templates/frr_bgp.j2
```

This is where the route reflector magic happens. The template iterates over the pre-computed neighbor list and adds `route-reflector-client` only when the current device is an RR

and the neighbor is not. The render engine already filtered the neighbor list, so the template does not need to know about device roles.

frr_vrf.j2

```
cat generators/python/templates/frr_vrf.j2
```

VRF BGP EVPN configuration with route distinguishers and route targets per VRF. The VXLAN dataplane configuration (vxlan interfaces, bridge mappings, anycast gateways) is included as reference comments because it is platform-specific. On a real EVPN switch these commands create the tunnels and gateways. In our FRR lab environment, the routing plane is what matters and the dataplane would be configured through Linux bridge and ip commands at the OS level.

Part 3: Path B – Ansible

The Ansible path lives in `generators/ansible/`. It generates the same FRR configurations using Ansible's inventory, group variables, and per-role templates instead of a Python render engine.

Install Ansible

On the VM:

```
sudo apt install -y ansible-core
```

Verify:

```
ansible --version
```

How It Is Structured

```
ls generators/ansible/
```

The key files are:

`inventory.yaml` defines every device with its loopback, ASN, fabric links, and a `role_template` variable that selects which Jinja2 template to render. Each device carries all the data it needs, mirroring what the Python render engine computes from the data model.

`group_vars/all.yaml` holds fabric-wide settings: OSPF parameters, BGP timers, MTU defaults, description prefix, and the `route_reflector_peers` list that non-RR devices use for their BGP neighbor statements.

`templates/` contains three templates: `spine.conf.j2`, `leaf.conf.j2`, and `border.conf.j2`. The spine template iterates over all hosts in the inventory and adds `route-reflector-client` for non-RR peers. The leaf and border templates peer only with the route reflectors listed in `route_reflector_peers`.

The playbook runs as `connection: local` and writes configs to `configs-ansible/` rather than pushing to devices. This keeps it self-contained.

Run It

```
cd ~/network-as-code-labs/generators/ansible
ansible-playbook -i inventory.yaml playbook.yaml
```

You should see all six hosts return changed status. Check the output:

```
ls ../../configs-ansible/
cat ../../configs-ansible/spine1.conf
cat ../../configs-ansible/leaf1.conf
```

Spine1 has `bgp cluster-id`, five neighbors, `route-reflector-client` on all non-RR peers. Leaf1 peers only with the two spines. Same logical config as the Python path.

Key Difference from Python Path

The Python path pre-computes everything in the render engine and hands the template a flat context. The Ansible path pushes more logic into the template because Ansible's variable system (hostvars, group membership) is the mechanism for role-based differentiation. Neither approach is wrong. The Python path is more explicit and easier to test. The Ansible path leverages existing conventions that Ansible teams already know.

Part 4: Path C – Terraform

The Terraform path lives in `generators/terraform/`. It demonstrates the declarative/stateful approach and is where the concept of “plan before apply” becomes tangible.

Install Terraform

On the VM, download the binary directly (the HashiCorp APT repo has GPG key issues on Ubuntu 24.04):

```
wget https://releases.hashicorp.com/terraform/1.12.1/
      terraform_1.12.1_linux_amd64.zip
sudo apt install -y unzip
unzip terraform_1.12.1_linux_amd64.zip
sudo mv terraform /usr/local/bin/
rm terraform_1.12.1_linux_amd64.zip LICENSE.txt
```

Verify:

```
terraform version
```

Run It

```
cd ~/network-as-code-labs/generators/terraform
terraform init
terraform plan
```

The `plan` output is the key demo moment. Terraform shows you every config file it will create and the exact content of each one. On the first run, everything is a “create” because there is no prior state. After applying, subsequent plans show only what changed.

```
terraform apply -auto-approve
```

Six config files land in `configs-tf/`. Check the output:

```
ls ../../configs-tf/  
cat ../../configs-tf/spine1.conf
```

The State Tracking Demo

This is what makes Terraform different from the other two paths. After the initial apply, run `plan` again:

```
terraform plan
```

Output: “No changes. Your infrastructure matches the configuration.” Terraform knows nothing changed because it tracks state.

Now modify a variable. Open `main.tf` and change `bgp_keepalive` from 30 to 60:

```
terraform plan
```

Terraform shows you exactly which config files will change and what the diff looks like. Every device config that references the BGP keepalive timer shows up as “must be replaced.” This is the plan/apply workflow that network engineers are used to from change management, expressed as code.

Revert the change after the demo.

What Terraform Adds

The `variable "devices"` block maps the topology data model into Terraform’s type system. The `locals` block derives route reflector and non-RR device sets, mirroring the Python render engine’s context building logic. The `templatefile()` function renders the `.tftpl` template using Terraform’s own template syntax (`%{~ if }`, `%{~ for }`) which is similar to Jinja2 but with different delimiters.

In a real Cisco NaC deployment, you would use vendor-specific Terraform providers that manage device configuration as actual resources with full plan/apply/destroy lifecycle. The `local_file` approach here demonstrates the pattern without requiring vendor-specific infrastructure.

Part 5: The Route Reflector Demo

This demo proves the value of intent-based config generation. You will temporarily remove `spine2` as a route reflector, regenerate configs, inspect the changes, then add it back. This shows what happens when the data model changes and you re-run the generator.

Step 1: Check the Current State

Before changing anything, look at how many BGP neighbors leaf1 currently has:

```
grep "neighbor.*remote-as" configs/leaf1.conf
```

You should see two lines: one for spine1 (10.0.0.1) and one for spine2 (10.0.0.2). Leaf1 peers with both route reflectors.

Now check spine1's route-reflector-client lines:

```
grep "route-reflector-client" configs/spine1.conf
```

Four lines: leaf1, leaf2, border1, border2. Spine2 is not a client because it is also an RR.

Step 2: Remove spine2 as a Route Reflector

Edit data/topology.yaml and change spine2's route_reflector flag from true to false:

```
- name: spine2
  role: spine
  loopback: 10.0.0.2/32
  management_ip: 172.20.20.102/24
  asn: 65000
  route_reflector: false    # was true
```

Also edit data/overlay.yaml and remove spine2 from the route_reflectors list:

```
route_reflectors:
- device: spine1
  cluster_id: 10.0.0.1
# spine2 entry removed
```

Step 3: Run Validation

```
uv run python validate.py
```

This should fail at SEM-03 because the topology still has spine2 not flagged as RR but the overlay no longer lists it. That is the validation framework doing its job. It caught the inconsistency.

Actually, since we changed both files consistently (topology flag is false, overlay list has it removed), validation should pass. Run it and confirm all 39 checks pass.

Step 4: Regenerate Configs

```
uv run python -m generators.python.render
```

Step 5: Inspect What Changed

Check leaf1's BGP neighbors again:

```
grep "neighbor.*remote-as" configs/leaf1.conf
```

Now there is only one line: spine1 (10.0.0.1). Leaf1 no longer peers with spine2 because spine2 is no longer a route reflector, and non-RR devices only peer with RRs.

Check spine1's route-reflector-client lines:

```
grep "route-reflector-client" configs/spine1.conf
```

Now there are five lines instead of four. Spine2 is now a client of spine1 because it is no longer an RR itself.

Check spine2's config:

```
grep "neighbor.*remote-as" configs/spine2.conf
```

Spine2 now has only one neighbor: spine1. It is an RR client, not an RR. It no longer peers with every device in the fabric.

Step 6: Count the Impact

One flag change in topology.yaml and one line removed from overlay.yaml. The generator updated configs for all six devices. Spine1 gained a client. Spine2 lost its RR role and all its client peerings. Every leaf and border dropped one neighbor. You touched two YAML files and the automation rewired the entire overlay.

Step 7: Revert

Change spine2 back to route_reflector: true in data/topology.yaml. Add the spine2 entry back to data/overlay.yaml:

```
route_reflectors:
- device: spine1
  cluster_id: 10.0.0.1
- device: spine2
  cluster_id: 10.0.0.2
```

Regenerate to restore the original configs:

```
uv run python validate.py
uv run python -m generators.python.render
```

Verify leaf1 is back to two neighbors:

```
grep "neighbor.*remote-as" configs/leaf1.conf
```

Two lines again. Everything is back to normal.

Part 6: Commit the Config Generators

The route reflector demo is reverted, validation passes, and all three config generation paths are working. Time to commit.

```
git status
```

You should see the `generators/` directory as untracked, plus the build guide. The data model files should be clean (no modifications) since you reverted the RR demo changes.

```
git add generators/ docs/lab3-build-guide.md
git status
```

Review the staged files. You should see the Python render engine and Jinja2 templates, the Ansible playbook and templates, and the Terraform configuration. The generated configs in `configs/`, `configs-ansible/`, and `configs-tf/` should not appear because the `.gitignore` excludes them.

```
git commit -m "Lab 3: config generation with Python/Jinja2,
              Ansible, and Terraform paths"
```

Check the log:

```
git log --oneline
```

Three commits. The data model, the validation framework, and now the config generation engine. Each commit represents a complete, working layer of the automation stack. If you need to go back to a point before config generation existed, the commit history gives you that option.

Part 7: Comparing the Three Paths

Now that all three paths have produced configs, prove they are equivalent. The Python path generates the most complete output (it includes VRFs, VXLAN segments, and host-facing interfaces). The Ansible and Terraform paths focus on the core routing config (OSPF + BGP) to keep the templates readable for comparison. The BGP peering logic is identical across all three.

Side-by-Side BGP Comparison

Extract the BGP section from each path's spine1 config:

```
grep -A 30 "router bgp" configs/spine1.conf | head -35
grep -A 30 "router bgp" configs-ansible/spine1.conf | head -35
grep -A 30 "router bgp" configs-tf/spine1.conf | head -35
```

All three show the same structure: ASN 65000, cluster-id 10.0.0.1, five neighbors, route-reflector-client on the four non-RR peers, both address families activated.

When to Use Which

Python + Jinja2 gives you the most control. The render engine is plain Python with no framework overhead. You can add arbitrary logic during context building and write tests against the generated output. The downside is you build everything yourself.

Ansible gives you role-based organization, inventory management, and a large ecosystem of modules. If your team already runs Ansible for server configuration, extending it to network devices is a natural fit. The downside is template logic can get complex and debugging Jinja2 inside Ansible is harder than debugging plain Python.

Terraform gives you state tracking and plan/apply workflow. You see diffs before applying changes, which maps directly to how change management works in most organizations. The downside is Terraform's templating is more limited than Jinja2, and the real power comes from vendor-specific providers rather than the `local_file` approach shown here.

For this lab series, the Python path is the primary path because it integrates cleanly with the validation framework from Lab 2 and the deployment pipeline in Lab 4. The Ansible and Terraform paths are here so you can run them, see the output, and make an informed decision about which tool fits your environment.

Part 8: What We Proved

By the end of this lab you have demonstrated three things.

First, that intent-based configuration generation eliminates manual per-device editing. Six devices with different roles, different interfaces, different VRF assignments, and different BGP peering all generated from a single data model. The configs are consistent because they come from the same source. There is no possibility of a typo on one device that does not exist on another because a human edited them independently.

Second, that the route reflector auto-peering pattern works. Declaring a device as a route reflector is a single flag in the data model. The render engine derives all the neighbor statements, client designations, and cluster-id configuration automatically. This is the pattern the Cisco NaC paper highlights as a key benefit of intent-based automation.

Third, that the same intent can be realized with different tools. Python, Ansible, and Terraform all produce equivalent configurations from the same data model. The choice of tool depends on your team's existing skills and infrastructure, not on the data model design. The data model is tool-agnostic.

Troubleshooting

Config generation fails with “missing components”: This means the data model has validation errors. Run `uv run python validate.py` first and fix any failures.

Templates produce blank sections: Check that the Jinja2 environment has `trim_blocks=True` and `lstrip_blocks=True`. Without these, blank lines accumulate from template control structures.

Leaf config has no VRFs: The render engine only assigns VRFs to leafs that have interface assignments with matching VLANs. If a leaf has no interfaces defined in `data/services/interfaces.yaml`, it gets no VRFs. This is intentional: a leaf with no host-facing ports does not need VRF configuration.

Ansible playbook fails with “command not found”: Install `ansible-core` with `sudo apt install -y ansible-core`. The playbook uses only built-in modules (`ansible.builtin.file` and `ansible.builtin.template`), so no extra collections are needed.

Ansible warns about Python interpreter: This is safe to ignore. Ansible discovers the system Python automatically. The warning is about future compatibility, not a current problem.

Terraform “command not found”: Follow the install steps in Part 4. Terraform is not included in most Linux distributions by default and needs the HashiCorp APT repository.

Terraform plan shows all files as “create”: On the first run, there is no state, so everything is a create. After `terraform apply`, subsequent plans show only what changed. This is the expected behavior and a good demo moment.

Terraform state file in the repo: The `.gitignore` already excludes `*.tfstate`, `*.tfstate.backup`, `.terraform/`, and `.terraform.lock.hcl`. If you see these in `git status`, check that your `.gitignore` is up to date.